本地知识库 · 面试问答

- **项目：** 基于 RAG 的本地文档知识库桌面客户端（Tauri 2 + React + Rust）
- **版本：** 1.0.0
- **整理日期：** 2026-09-07
- **口径：** 答案对齐仓库实现，不把设计意图说成已经防住；没测过的数字不编。

一、项目介绍

问： 先讲一下这个项目。

答： 这是一个**本地文档知识库桌面客户端。**把 PDF、Markdown、TXT、Word 放进本机，之后用自然语言问，答案要能指回原文片段，库里没有依据就拒答，而不是硬编。

产品定位是个人知识管理，不是云 RAG。边界划死：**文档、分块、嵌入、检索索引全部留在本机**；只有「怎么把答案写出来」可以切到 SiliconFlow。嵌入默认走本机 Ollama（nomic-embed-text），换模型要整库重嵌，所以嵌入不上云——一上云，隐私承诺就从「本地库」变成「把资料交给别人」。

技术上是 Tauri 2 的混合架构。Rust 管 SQLite、文件、Ollama/Chroma 进程和 IPC；TypeScript 管解析、分块、混合检索和 Prompt 编排；React 做界面。

- **导入链路：** 解析 → 分块 → SQLite（含 FTS5）→ 本地嵌入 → 写入 Chroma
- **问答链路：** 向量 + 关键词混合检索 → RRF 融合 → 弱证据拒答 → 按上下文预算拼 Prompt → 流式生成 → 分块级引用

重排默认 RRF：稳、快、能单测、能进 CI。LLM 列表重排做成可选，失败则 fail-open 退回 RRF。

刻意补了两块面试里最容易被打的地方：

1. **可排查：** 检索工作台和分块预览，能分清是「搜错了」还是「答错了」。
2. **可回归：** 离线 eval 用 mock 向量盯融合和门控，进 CI；本机还有一轮真 Ollama 嵌入小样本，18 题里可答题 Hit@5 为 15/15，MRR 约 0.889，3 道域外题拒答正确。数字只当回归基线，语料只有 5 个块，不拿它当生产指标。

当前是 1.0，Windows 安装包能装能跑。短板也很清楚：检索评测是小样本，live 路径不经过 Tauri/Chroma；SiliconFlow 只加速生成，Key 落在本机 SQLite，防的是前端把完整 Key 打进日志或网络面板，防不了本机有权限的进程直接读用户目录。

二、实现与取舍（连环问）

2.1 文档、分块、嵌入、索引为什么必须本机？生成为什么可以上硅基流动？

本地知识库的承诺是：**资料是什么、索引里有什么，不能离开这台机器。**文档原文、分块、Ollama 嵌入、SQLite/FTS、Chroma
都是「库」本身；换嵌入模型还要整库重嵌，嵌入一上云，威胁模型就从本地库变成云 RAG。

生成可以切
SiliconFlow，是因为「答案怎么组织」和「库里有什么」可分离。云端只增强写答案，不替代存和检。

2.2 问句本身算出境吗？Prompt 里拼了原文片段，生成走云等于检索证据出过境。

算。SiliconFlow 模式下出站的是整包 chat messages：用户问句 + 系统提示
+「参考资料」里的**原文片段**。问句本身出境；检索证据也出境。

正确边界是：

- **仍在本机：** 原文件、SQLite 分块/FTS、嵌入向量、Chroma
- **会出境：** 这次问答的 query、拼进 Prompt 的 chunk 文本、对话历史

不能说成「只有模型在云、数据全在本地」。更准确是：**索引和语料库在本地；一旦开云生成，这一轮用到的问句和检索证据会离开本机。**

2.3 Key 放 SQLite、走 Rust 出站，防的是什么？防不了什么？

防的是 WebView / 前端脚本 / 网络面板把完整 Authorization: Bearer 打出来。get_setting / get_all_settings 对 siliconflow_api_key 只返回掩码；真正带 Key 的请求由 Rust 读库后代理。

防不了：本机已有用户权限的进程直接读应用数据目录里的 SQLite。这是桌面应用的普遍威胁，要 OS 密钥环才能再升一档，当前没做。

2.4 Tauri 里 Rust 和 TS 怎么分工？检索决策为什么在 TS 不在 Rust？

- **Rust：** SQLite、FTS、文件、Chroma/Ollama 进程、SiliconFlow 出站代理
- **TypeScript：** 解析、分块、混合检索编排、RRF、门控、Prompt、流式对话

检索决策放 TS，是因为 rerank、shouldDeclineAnswerDueToWeakEvidence、token 预算都是纯函数，能单测，离线 eval 不用起 Tauri/Chroma 就能回归融合和门控。若全写进 Rust，CI 和面试演示都绑死原生栈。

2.5 800、100、50 的单位是什么？是 tiktoken 还是启发式？

都是启发式 token，不是 tiktoken。estimateTokenCount：汉字 +1，空白跳过，其它字符 +0.25，再 ceil。默认
max_chunk_size=800、min_chunk_size=100、overlap=50，都走这套计数。

2.6 为什么先按标题切，再按长度？代码为什么单独切？

先按标题/章节切，是为了让块带着
heading_path，检索和引用有结构，而不是切成无语义的滑窗。超长再按段落切；代码走 chunkCode，按行累加，避免把函数从中间撕开后和普通段落合并。实现上：代码块不和过小的文本块 merge，从非代码到代码也不做 overlap 粘贴。

2.7 overlap 50 相对 800 很短，骑缝答案会不会漏？

会有残余风险。50/800 大约
6%，只能覆盖切点附近一小截。长句、表格行、关键数字若刚好跨块且超过约 50 个启发式 token，下一块只带得上尾巴。标题优先能减少「同节被拦腰砍」的次数，但不能保证。这是延迟和重复索引之间的权衡，不是骑缝免疫。

2.8 PDF 为什么在主线程？扫描件没有 OCR，导入成功但检索全空，算不算 bug？

PDF.js 自己已经有 Worker。再套一层 parse-chunk Worker，在 Tauri WebView 里容易嵌套 Worker / 路径 / 传输踩坑。所以 PDF（以及当前实现里的 DOCX）留在主线程；md/txt 才进 Worker。

扫描件：只抽 getTextContent，没有 OCR。没字就几乎没有 section，可能 0
块仍标成「就绪」。这不是崩溃型 bug，是**能力缺口 + UX**
缺口：导入没失败，但索引是空的。产品上应该警告「未抽出文本」，现在没有强拦。

2.9 char_start 是 PDF 页坐标还是拼接后的偏移？高亮对不齐怎么解释？

是分块内容依次拼接后的字符偏移，不是 PDF 页坐标，也不是 buildFullDocumentText(sections) 的下标。overlap 会改
content，偏移跟着变。预览高亮用的是在全文里搜块文本（findHighlightInFullText），不能拿裸 char_start 去切原文。对不齐通常来自：PDF 抽字时空白被折叠、预览又做了 section 合并、或 overlap 前缀出现在块里但不在「干净原文」里。

2.10 入库 search_document:、查询 search_query:，只加一边会怎样?

nomic-embed-text 是非对称前缀。只加一边，query 和 document
不在同一指令空间，余弦会系统性偏弱，表现为「明明相关却像库外」。两边都不加，能用，但比按模型卡加前缀差。只对模型名包含 nomic-embed 的才加。

2.11 768 维、一块一向量，一千块大概多大？百万文档扛不扛?

nomic-embed-text 是 768 维 float32。一千块原始向量大约 1000×768×4 ≈ 3MB，再加上 SQLite 原文和 Chroma 的 HNSW/元数据，量级是数 MB 到十几 MB，个人库没问题。

百万文档扛不住。按每篇几十块算是千万级向量，仅向量就是几十 GB 起，本机 Chroma
进程不是这个规模的产品假设。这是单用户桌面库，不是企业检索集群。

2.12 批次 5、失败退避 3 次，一篇里部分块嵌失败：FTS 有原文、Chroma 没向量，检索会表现成什么?

SQLite/FTS 先落全文；Chroma 只在某批嵌入成功后才
chromadb_add_documents。部分失败时文档仍标
ready，并带「部分分块向量化失败（x/n）」。

表现：关键词能搜到失败块，语义搜不到。混合模式可能只靠 FTS
召回它们；纯语义会像这些块不存在。全失败才标 error。

2.13 SQLite 和 Chroma 谁是真相？删库删不干净会搜到幽灵文档吗?

SQLite 是文档/分块是否存在的真相；Chroma 是派生向量索引。

会幽灵。删文档时 SQLite 会删，Chroma 删除是 let _ =
chroma.delete_documents(...)，失败被吞。于是向量腿还能搜到已删内容。反过来，部分嵌入失败是 FTS 有、Chroma 无。两边都不是自动双写事务。

2.14 Chroma 回的是 distance，你怎么变成门控用的相似度？换 L2/cosine，0.46 还能用吗?

distancesToSimilarities：若 |d| 的最大值 ≤ 1.05，按余弦距离 sim = 1 - d；否则按 1/(1+|d|) 当 L2 启发式。门控阈值 0.46 是转换后的相似度，是对着当前这条启发式 + Nomic 经验定的。

换度量或换 Chroma space，0.46 **不能原样用**。数字会漂，必须重标定。创建 collection
时也没有把 hnsw:space 写死成 cosine。

2.15 向量和 BM25 为什么不能加权加分?

量纲不同。向量相似度大约在 0～1；BM25/-bm25(fts) 无上界。直接 α·sim + β·bm25 会被关键词分淹没，权重失去可比性。所以用名次融合，不拿原始分相加。

2.16 公式能不能写出来：α/(k+r_vec) + β/(k+r_kw)，k=60、α/β=0.7/0.3 从哪来?

实现是加权 RRF（0-based 名次 i）：

```
s = α/(k + r_vec) + β/(k + r_kw)
r = i + 1
k = 60
```

k=60 是 Cormack 等人 RRF 的常用常数，对长尾名单次不敏感。α/β 默认 0.7/0.3，再归一化成 α+β=1，是产品默认「语义为主、关键词托底」，可在设置改，不是理论最优解。之后还会加标题 γ 和文件名 lexical bonus。

2.17 只在 FTS 一路排第一、向量很糊，还能不能进 top 6?

能进融合池，这正是 RRF 要保护的：单路第一名仍有 β/(k+1)，不会因为另一路缺席被乘成 0。

是否稳进最终 top 6，还看向量池有多长、以及标题/文件名 bonus。向量池默认可到几十条，纯 RRF 下 FTS-only 第一名不一定压过向量第 6 名；文件名命中那种 0.42 量级的 bonus 往往会把它托上去。所以「能」是机制保证有贡献，不是数学上必进前六。

2.18 标题 γ=0.12、文件名还加 bonus，会不会「文件名碰巧包含查询词」就刷上去?

会。heading_path 按查询词命中比例加 γ=0.12；lexicalRelevanceBoost 里文件名整句包含查询直接 +0.42，词项命中再按比例加。文件名叫 hybrid-retrieval.md、查询带 hybrid，会明显抬分。这是为了纠正向量漂到无关片段，副作用就是文件名撞词会刷榜。工作台能看出来。

2.19 单文档最多占 top 6 里几条？这是 MMR 还是硬 cap?

硬 cap，不是 MMR。maxPerDocument = max(2, ceil(maxResults × 0.55))，默认 top 6 时是 **4 条**。先按 cap 选，不够再回填，回填阶段同一文档还可以再进。MMR 是按相似度罚多样性；这里只是计数上限。

2.20 FTS 中文按字 OR、没 jieba，「红烧肉怎么做」这种域外题会不会被常用字命中?

会。查询被拆成单字（红/烧/肉/怎/么/做）再 OR，没有 jieba。常见汉字会在技术文档里假命中。所以**不能**「FTS 有行就生成」——有行只说明字面召回，不说明能答。

评测里 3 道拒答题故意用英文域外句，就是为了躲开这套 CJK OR。中文口语域外题，这条腿比英文脏。

2.21 混合模式具体怎么拒?

shouldDeclineAnswerDueToWeakEvidence：

- 一个候选都没有 → 拒
- 语义模式：向量顶分 < 0.4 → 拒
- 关键词模式：有候选就不因向量拒（没候选才会在上游变成空）
- **hybrid：向量顶分 < 约 0.46 且 keywordCandidateCount === 0 才拒**

注意：keywordCandidateCount 是 FTS 池大小，不是重排后的 top 6。

2.22 只有一路很强、另一路全空，会不会误拒或误答?

会两种都发生。

- 向量强、FTS 空：不拒。同义改写靠这个活。
- 向量弱、FTS 有行：不拒。专有名词、向量没嵌上的块靠这个活。
- 向量弱且 FTS 空：拒。减轻库外胡编。
- FTS 被常用字刷出行、向量也糊：hybrid **不拒**，这是误答通道。「红烧肉」就走这里。

2.23 拒答是不调生成模型，还是调了再让模型说不知道?

不调。门控为真就直接吐固定拒答文案，return，不 streamChat。自检是生成之后的可选第二跳，跟拒答不是一条路。

2.24 引用是模型自己编的，还是 Prompt 里的块序号点回原文?

Prompt 把检索结果编成 [文件名#1]... 这种块，要求模型按这个格式引用。UI 解析 [文档名#序号]，用序号去对**这一轮真正检索到的 references 数组**（chunkIds[refIndex-1]）。

所以可点回的是「Prompt

里的块」，不是模型在库外发明的新文档。模型仍可能写错序号、写错文件名；点到的仍是数组里那一项。没有做「引用句是否真在该块中」的对齐校验。

2.25 LLM 重排 / 贴地自检失败为什么 fail-open？代价是什么？

重排：解析失败、id 不全、抛错，原样返回 RRF 顺序，聊天不中断。
自检：额外一次非流式 JSON；解析失败或调用失败，不加「自检提示」，主回答照发。

代价：自检失败 = 没检。重排失败 =
没神经重排。用户看到的是「有回答」，不是「系统拒绝」。换来的是可用性，换掉的是这层保险。

2.26 15 库内 + 3
域外，题是谁出的、对着哪几篇文档？会不会题和块一一对应，测的是记忆不是检索？

题是项目自己写的 evals/golden.json，对着 evals/fixtures/mini-kb
五篇短文档：ollama-setup、hybrid-retrieval、siliconflow-privacy、chunking-strategy、chromadb-lifecycle。很多是抽数字（11434、800、0.7），和块几乎一一对应。

这套题测的是融合、门控、前缀、回归是否跑通，**不是难检索、不是独立标注的金标。**小样本已经接近饱和，Hit@5=15/15 不能当泛化能力。3 道拒答还是英文域外，避开了中文 FTS 假命中。

2.27 离线 CI 用假向量，live 才走 Ollama；离线 MRR 0.967、live 0.889，差在哪？

离线：词袋余弦当假向量 + 词项重叠当关键词，和出题用词高度同分布，排第一很容易。
Live：真 nomic-embed-text，语义空间不再等于词面重合，MRR 掉一截是预期。Live
关键词腿仍是 harness 的 term-overlap，也不是应用里的 FTS5。

所以 0.967 → 0.889 主要差在「假向量太贴题」vs「真嵌入会排错文件」（例如 diversity 那题 live 会先打到 chunking-strategy）。

2.28 live 评测绕开 Tauri/Chroma——CI 保的是什么？

必须说清楚：eval:retrieval 进 CI，保的是 rerank / gate / overlap 这些纯函数不回归。Live
证明真嵌入空间上这 18 题还能命中。

不证明： 桌面 IPC、chromadb_query、distance 转换、FTS5 按字
OR、导入失败、删库幽灵。那些只有装起来点过才算。

2.29 LLM 重排开了 Hit@5 还是 15/15，为什么默认关?

小样本已经饱和，开了没有更好的 Hit@5/MRR，只多了一次 listwise 调用：慢、依赖对话模型、输出不可复现。默认 RRF 是工程选择：稳、可单测、能进 CI。设置里可开，失败 fail-open。没在更大语料上证明它值得当默认。

2.30 没有 query rewrite、没有 cross-encoder，漏召回时你怎么查？检索工作台拆的是「搜错了」还是「答错了」？

没有改写，没有 cross-encoder。漏了就用人肉拆链路：检索工作台只跑 retrieve()，展示向量分、关键词分、融合后的块，**不调生成。**对话页才生成，来源面板展示实际进 Prompt 的引用。

- 工作台 top 就错：搜错了（分块、前缀、FTS、权重、文件名 bonus、硬 cap）
- 工作台对、回答胡编：答错了（Prompt、模型、云生成、自检 fail-open）
- 扫描 PDF 空文本、部分嵌入失败：更早，索引就不完整

这是用可观察性补模型能力，不是用更重的检索模型把召回补上。

三、RAG、提示词与用 AI 做项目

3.1 讲解一下 RAG

RAG 是 Retrieval-Augmented Generation：先从你的资料里找出可能有用的片段，再让模型**只根据这些片段**生成回答。它解决的是模型没读过你的私有文档、而且会用流畅的常识去补全。

拆开就三步，缺一不可：

1. **索引：** 文档 → 解析 → 分块 → 嵌入 → 写入向量库，同时原文进 SQLite/FTS。
2. **检索：** 问句变成向量，和关键词一起搜，融合、截断、必要时拒答。
3. **生成：** 把选中的块塞进 Prompt，要求按块作答并带引用。

这个项目里，RAG 的产品核心在**检索和门控**，不在模型是否更会写。混合检索、RRF、0.46 门槛、引用序号，都是在管「模型看见什么」。生成可以走本地 Ollama 或 SiliconFlow，换的是写答案的人，不是库。

一句话：**没有可靠检索的 RAG，只是把幻觉从参数里搬到了 Prompt 里。**

3.2 说一下你理解的提示词工程

提示词工程不是堆形容词，是给模型设**约束、格式、证据范围、失败时怎么办**，并且用评测盯住它有没有遵守。

这个项目里写过三类，角色完全不同：

- **主问答：** 系统提示写死「只能根据参考资料」「没有就说没有」「引用写成 [文档名#序号]」。这是行为合同，不是文采。
- **LLM 重排：** 只许输出 chunk_id 的 JSON 数组，解析失败就 fail-open 回 RRF。
- **贴地自检：** 再要一次 {"grounded":boolean}，失败当没检，不阻断用户。

有效的提示词通常具备：角色清楚、禁止项可检查、输出可解析、和检索结果绑在一起。没有检索证据时，再精的提示词也只是让模型更礼貌地编。

也不迷信「换一句就能质变」。这个项目里，拒答是代码门控，不是靠模型自觉说不知道。提示词管生成形态；**能不能答，是检索说了算。**

3.3 上下文怎么管理

上下文是有限预算，不是聊天记录越长越好。这个项目按 **8192 启发式 token** 切成四块：系统 10%、参考资料 60%、历史 20%、当前问句 10%。资料权重大，因为 RAG 的答案质量取决于块，不取决于上次闲聊。

具体做法：

- 历史只留最近 **6 轮**用户回合，再按 token 从后往前截。
- 检索最多 6 块，超预算就截断块内容，并标「内容已截断」。
- 问句本身也截断，避免超长粘贴把资料挤掉。
- 自检另开一次短调用，资料摘要、问句、回答都有字符上限，避免质检自己先爆窗。

还有一层常被漏掉的：检索上下文和对话上下文不是一回事。分块 800、overlap 50 是**索引时的上下文**；Prompt 里的 top 6 是**生成时的上下文**。切坏了，后面预算再大方也救不回来。

云生成时还要意识到：进 Prompt 的问句和原文片段都会出境。上下文管理同时是质量和隐私问题。

3.4 写过 skill 吗

这个知识库仓库里**没有** SKILL.md，没有把它封装成可安装的 Agent Skill。

但写过同一类东西：给执行者的、带触发条件和边界的操作说明。这个项目的 docs/superpowers/specs、分阶段 plans、技术取舍一页纸、评测 golden.json，本质上就是

skill——告诉人和
AI「现在做哪一步、用什么前提、不要提前引入后面的细节」。问答侧的系统提示、重排 JSON 协议、fail-open 规则，也是写给模型的 skill。

如果现在要写 Cursor
Skill，会按这个结构：什么时候触发、先读哪份事实、步骤清单、明确禁止编造的数字和越权改文件。Skill 的价值是**可复用的约束**，不是把聊天记录存下来。

3.5 使用 AI 时最重要的是什么

最重要的是**可验证**，不是生成速度。

模型擅长把不确定说得很顺。这个项目里把验证做成了几层：弱证据不调生成、引用必须点回检索数组、检索工作台拆开「搜错了还是答错了」、纯函数单测、离线 eval 进 CI。没有这些，AI 写的 RAG 看起来能聊，其实在用常识答题。

落地时三条：

1. 先有对错标准（golden、门控、类型、能跑的安装包），再让 AI 写代码。
2. AI 给的数字、API、安全结论，对不上仓库就不说。
3. 失败要 fail-open 并承认「这层没生效」，不能假装检过了。

用 AI 的核心能力是：**会不会设计检查点，以及敢不敢说「我没测过」。**

3.6 用 AI 实现项目的步骤

做这个 1.0 的顺序，不是「先让它生成一个能聊的页面」。

1. **先定边界：** 本地库承诺什么、嵌入为什么不上云、Key 防什么不防什么。没有这一步，AI 会默认做成全云 RAG。
2. **先写设计再写计划：** 架构、数据模型、阶段依赖写在 spec/plan 里，一期只做一期。解析、嵌入、检索、对话是串起来的，不能让 AI 平行乱铺。
3. **先可测的纯函数：** 分块 overlap、RRF、门控、引用解析、上下文预算。这些 AI 写得快，也必须先有测试，否则后面全是不可回归的「感觉更好」。
4. **再接原生和外部进程：** SQLite、FTS、Ollama、Chroma、Tauri IPC。AI 容易在这里编 API，要对着真实命令和失败路径改。
5. **再接生成：**
Prompt、流式、拒答、可选云端。生成放后面，是因为没有检索的对话没有产品意义。
6. **用评测和工作台收口：** 离线 eval 进 CI，live
小样本看真嵌入，工作台给人拆搜错/答错。最后才是安装包和设置打磨。

用 AI

加速的是「按计划实现和补测试」，不是「跳过设计直接出仓库」。每一步都要留下它不能抵赖的证据：测试、评测、或自己点过的界面。